

RULEBERT: Teaching Soft Rules to Pre-Trained Language Models

Mohammed Saeed[†] Naser Ahmadi[†] Preslav Nakov[‡] Paolo Papotti[†]
[†]Eurecom, France [‡]Qatar Computing Research Institute, HBKU, Qatar
 { *FirstName.LastName* }@eurecom.fr, pnakov@hbku.edu.qa

Abstract

While pre-trained language models (PLMs) are the go-to solution to tackle many natural language processing problems, they are still very limited in their ability to capture and to use common-sense knowledge. In fact, even if information is available in the form of approximate (soft) logical rules, it is not clear how to transfer it to a PLM in order to improve its performance for deductive reasoning tasks. Here, we aim to bridge this gap by teaching PLMs how to reason with soft Horn rules. We introduce a classification task where, given facts and soft rules, the PLM should return a prediction with a probability for a given hypothesis. We release the first dataset for this task, and we propose a revised loss function that enables the PLM to learn how to predict precise probabilities for the task. Our evaluation results show that the resulting fine-tuned models achieve very high performance, even on logical rules that were unseen at training. Moreover, we demonstrate that logical notions expressed by the rules are transferred to the fine-tuned model, yielding state-of-the-art results on external datasets.

1 Introduction

Pre-trained language models (PLMs) based on transformers (Devlin et al., 2019; Liu et al., 2020) are established tools for capturing both linguistic and factual knowledge (Clark et al., 2019b; Rogers et al., 2020). However, even the largest models fail on basic reasoning tasks. If we consider common relations between entities, we see that such models are not aware of negation, inversion (e.g., *parent-child*), symmetry (e.g., *spouse*), implication, and composition. While these are obvious to a human, they are challenging to learn from text corpora as they go beyond linguistic and factual knowledge (Ribeiro et al., 2020; Kassner and Schütze, 2020). We claim that such reasoning primitives can be transferred to the PLMs by leveraging logical rules, such as those shown in Figure 1.

Input facts:

Mike is the parent of **Anne**. **Anne** lives with Mark. **Anne** is the child of Laure. **Anne** lives with *Mike*.

Input rules:

(r_1 , .1) Two persons living together are married.
 (r_2 , .7) Persons with a common child are married.
 (r_3 , .9) Someone cannot be married to his/her child.
 (r_4 , 1) Every person is the parent of his/her child.

Test 1: Laure and *Mike* are married.

Answer: *True* with probability 0.7 [r_4 , r_2]

Test 2: **Anne** and Mark are married.

Answer: *False* with probability 0.9 [r_1]

Test 3: **Anne** and *Mike* are married.

Answer: *False* with probability 0.9 [r_1 , r_3 , r_4]

Figure 1: Examples of hypotheses that require reasoning using facts and possibly conflicting soft rules (rule id and confidence shown in brackets).

While there have been initial attempts to teach reasoning with rules to PLMs (Clark et al., 2020; Kassner et al., 2020), such approaches model only a subclass of logical rules. In fact, current solutions focus on exact rules, i.e., rules that hold in all cases. In reality, most of the rules are approximate, or soft, and thus have a certain confidence of being correct. For example, across the 7,015 logical rules defined on the DBpedia knowledge graph, only 11% have a confidence above 95%. In the example, rules r_1 – r_3 are soft, i.e., cover knowledge that is not true in all circumstances. Consider rule r_2 , stating that if two persons have a child in common, they are most likely married. As r_2 has a confidence of being correct of 0.7, this uncertainty is reflected in the probability of the prediction.

With the above considerations in mind, here we show how to reason over soft logical rules with PLMs. We provide facts and rules expressed in natural language, and we ask the PLM to come up with a logical conclusion for a hypothesis, together with the probability for it being true.

Unlike previous approaches (Clark et al., 2020), we enable deductive reasoning for a large class of soft rules with binary predicates and an unrestricted number of variables. Our model can even reason over settings with conflicting evidence, as shown in Test 3 in Figure 1. In the example, as Anne and Mike live together, they have a 0.1 probability of being married because of soft rule r_1 . However, we can derive from exact rule r_4 that Anne is the child of Mike and therefore they cannot be married, according to soft rule r_3 .

To model uncertainty, we pick one flavor of probabilistic logic programming languages, LP^{MLN} , for reasoning with soft rules (Lee and Wang, 2016). It assigns weights to stable models, similarly to how Markov Logic assigns weights to models. However, our method is independent of the logic programming approach at hand, and different models can be fine-tuned with different programming solutions. Our proposal makes use of synthetic examples that “teach” the desired formal behavior through fine-tuning. In particular, we express the uncertainty in the loss function used for fine-tuning by explicitly mimicking the results for the same problem modeled with LP^{MLN} .

Our contributions can be summarized as follows:

- We introduce the problem of teaching soft rules expressed in a synthetic language to PLMs through fine-tuning (modeled as binary classification).
- We create and release the first dataset for this task, which contains 3.2M examples derived from 161 rules describing real common-sense patterns with the target probability for the task obtained from a formal reasoner (Section 4).
- We introduce techniques to predict the correct probability of the reasoning output for the given soft rules and facts. Our solution relies on a revised loss function that effectively models the uncertainty of the rules (Section 5). Our approach handles multi-variable rules and nicely extends to examples that require reasoning over multiple input rules.
- We show that our approach enables fine-tuned models to yield prediction probability very close to that produced by a formal reasoner (Section 6). Our PLM fine-tuned on soft rules, RULEBERT, can effectively reason with facts and rules that it has not seen at training, even when fine-tuned with only 20 rules.
- We demonstrate that our fine-tuning approach effectively transfers knowledge about predicate negation and symmetry to the lower levels of the transformer, which benefits from the logical notions in the rules. In particular, RULEBERT achieves new state-of-the-art results on three external datasets.

The data, the code, and the fine-tuned model are available at <http://github.com/MhmdSaaid/RuleBert>.

2 Related Work

PLMs have been shown to have some reasoning capabilities (Talmor et al., 2020b), but fail on basic reasoning tasks (Talmor et al., 2020a) and are inconsistent (Elazar et al., 2021), especially when it comes to negation (Kassner and Schütze, 2020).

Our work focuses on deductive reasoning. Note that it is different from previous work, e.g., on measuring the factual knowledge of PLMs (Petroni et al., 2019), on probing the commonsense capabilities of PLMs at the token or at the sentence level (Zhou et al., 2020), or on testing the reasoning capabilities of PLMs on tasks such as age comparison and taxonomy conjunction (Talmor et al., 2020a). Our work relates to Task #15 in the bAbI dataset (Weston et al., 2016) and to RuleTakers (Clark et al., 2020). However, we differ (i) by using a larger subclass of first-order logic rules (with more variables and various forms), and (ii) by incorporating soft rules.

Our proposal is different from work on Question Answering (QA) with implicit reasoning based on common-sense knowledge (Clark et al., 2019a), as we rely purely on deductive logic from explicitly stated rules.

Our approach also differs from methods that semantically parse natural language into a formal representation on which a formal reasoner can be applied (Liang, 2016), as we directly reason with language. Yet, we are also different from Natural Language Inference (NLI) and textual entailment, which work with text directly, but cannot handle Horn rules (MacCartney and Manning, 2009; Dagan et al., 2013).

Unlike previous work (Yang et al., 2017; Hamilton et al., 2018; Minervini et al., 2020), we do not design a new, ad-hoc module for neural reasoning, but we rely solely on the transformer’s capability to emulate algorithms (Wang et al., 2019b; Lample and Charton, 2020).

3 Background

Language Models. We focus on language models pre-trained with bidirectional transformer encoders using masked language modeling (Devlin et al., 2019). For fine-tuning, we create examples for sequence classification to teach the models how to emulate reasoning given facts and soft rules.

Logical Rules. We rely on existing corpora of declarative *Horn rules* mined from large RDF knowledge bases (KBs) (Galárraga et al., 2015; Ortona et al., 2018; Ahmadi et al., 2020). An RDF KB is a database representing information with triples (or *facts*) $p(s, o)$, where a *predicate* p connects a *subject* s and an *object* o . An atom in a rule is a predicate connecting two universally quantified variables. A Horn rule (or clause) has the form: $B \rightarrow h(x, y)$, where $h(x, y)$ is a single atom (head or conclusion of the rule) and B (body or premise of the rule) is a conjunction of atoms. Positive rules identify relationships between entities, e.g., r_1, r_2, r_4 in Figure 1. Negative rules identify contradictions, e.g., r_3 in Figure 1. Rules can contain predicates comparing numerical values, such as $<$. For example, negative rule r_5 : $\text{birthYear}(b, d) \wedge \text{foundYear}(a, c) \wedge <(c, d) \rightarrow \text{negfounder}(a, b)$ states that any person (variable b) with a birth year (d) higher than the founding year (c) of a company (a) cannot be its founder. A fact is derived from a rule if all the variables in the rule body are replaced with constants from facts. For r_5 , facts “foundYear(Ford, 1903), birthYear(E. Musk, 1971), $>(1971, 1903)$ ” trigger the rule that derives the fact $\text{negFounder}(\text{E. Musk}, \text{Ford})$.

Rule Confidence. *Exact* rules, such as r_4 , apply in all cases, without exception. However, most rules are approximate, or soft, as they apply with a certain likelihood. For example, r_3 in Figure 1 is true in most cases, but there are historical exceptions in royal families. Rules are annotated with a measure of this likelihood, either manually or with a computed *confidence* (Galárraga et al., 2015).

Probabilistic Answer Set Programming. As we deal with soft rules, we adopt LP^{MLN} (Lee and Wang, 2016) to create the dataset. LP^{MLN} is a probabilistic extension of answer set programs (ASP) with the concept of weighted rules from Markov Logic (Baral, 2010). In ASP, search problems are reduced to computing *stable models* (answer sets), a set of beliefs described by the program.

A weight (or confidence) is assigned to each rule, so that the more rules a stable model satisfies, the larger weight it gets, and the probability of the stable model is computed by normalizing its weight among all stable models. Given a set of soft rules and facts, we measure how much the hypothesis is supported by the stable model.

4 Dataset

We start by defining the reasoning task. We then discuss example generation methods for three scenarios: single rule as input, multiple (possibly conflicting) rules that require reasoning for the same conclusion, and multiple rules that require a sequence (chain) of reasoning steps. Examples of the data generation procedures are in the Appendix.

4.1 Reasoning Task

Each example is a triple (*context*, *hypothesis*, *confidence*). *Context* is a combination of rule(s) and generated facts, such as “If the first person lives together with the second person, then the first person is the spouse of the second person.” and “Anne lives with Mike.” *Hypothesis* is the statement to be assessed based on the context, e.g., “Laure is the spouse of Mike.” *Confidence* is the probability that the hypothesis is valid given by the reasoner, e.g., 0.7. As we generate the examples, we know the confidence for each hypothesis.

4.2 Single-Rule Dataset Generation

Given a rule, we generate examples of different hypotheses to expose the model to various contexts. Each example contains the context c and a hypothesis h with its probability of being true as obtained for the (c, h) pair from the LP^{MLN} reasoner. The intuition is that the examples show the expected behavior of a formal reasoner for every combination of possible facts for a given rule. This process is not about teaching the model specific facts to recall later, but teaching it reasoning patterns.

Unlike previous work (Clark et al., 2020), our rules allow for multiple variables. This introduces additional complexity as examples must show how to deal with the symmetry of the predicate. For example, $\text{child}(\text{Alice}, \text{Bob})$ and $\text{child}(\text{Bob}, \text{Alice})$ are not equivalent since *child* is not symmetric, while $\text{spouse}(\text{Alice}, \text{Bob})$ and $\text{spouse}(\text{Bob}, \text{Alice})$ are equivalent as *spouse* is symmetric. We assume that meta-data about the symmetry and the types is available from the KB for the predicates in the rules.